

1. Tạo bảng với 3 feature “brand”, “age_group” và “category”

Code:

```
import polars as pl
from datetime import datetime

def build_feature_label(
    transactions_lf: pl.LazyFrame, # purchase_df.lazy()
    items_lf: pl.LazyFrame,       # item_df.lazy()
    users_lf: pl.LazyFrame,       # chưa dùng, để đúng spec
    begin_hist: datetime,
    end_hist: datetime,
    begin_recent: datetime,
    end_recent: datetime,
) -> pl.LazyFrame:
    """
    Tạo bảng Feature - Label cho bài toán khuyến nghị top-k.

    Output columns:
    - customer_id
    - item_id
    - brand_counts
    - age_counts
    - category_counts
    - segment_counts          (# lần mua cùng (category_l1, segment_name) với
item hiện tại)
    - target_user_group_counts
    - Y
    """

    # 0. Làm sạch transactions_lf: bỏ cột segment_name_right nếu tồn tại (do join
trước đó sinh ra)
    tx_cols = transactions_lf.columns
    if "segment_name_right" in tx_cols:
        base_tx = transactions_lf.drop("segment_name_right")
    else:
        base_tx = transactions_lf

    # 1. Filter giai đoạn HIST và RECENT trên bảng giao dịch
    hist_lf = (
        base_tx
        .filter(
            pl.col("created_date").is_between(begin_hist, end_hist, closed="both")
        )
    )

    recent_lf = (
        base_tx
        .filter(
            pl.col("created_date").is_between(begin_recent, end_recent,
closed="both")
        )
    )

    # 2. Thuộc tính item cơ bản (từ item_df)
    item_attrs = items_lf.select([
        "item_id",
        "brand_final",
        "age_bucket_final",
```

```

        "category",
        "category_l1",
        "target_user_group_final",
    ])

# 3. Mapping item_id -> segment_name đại diện (mode trong HIST)
item_segment = (
    hist_lf
    .group_by("item_id")
    .agg(pl.col("segment_name").mode().alias("segment_name_list"))
    .with_columns(
        pl.col("segment_name_list").list.first().alias("segment_name")
    )
    .select(["item_id", "segment_name"])
)

# 4. HIST đã gắn đầy đủ: brand/age/category/category_l1/segment_name
hist_enriched = (
    hist_lf
    .join(item_attrs, on="item_id", how="left")
    .join(item_segment, on="item_id", how="left")
)

# 5. Feature 1: brand_counts
brand_counts = (
    hist_enriched
    .group_by(["customer_id", "brand_final"])
    .agg(pl.len().alias("brand_counts"))
)

# 6. Feature 2: age_counts
age_counts = (
    hist_enriched
    .group_by(["customer_id", "age_bucket_final"])
    .agg(pl.len().alias("age_counts"))
)

# 7. Feature 3: category_counts
category_counts = (
    hist_enriched
    .group_by(["customer_id", "category"])
    .agg(pl.len().alias("category_counts"))
)

# 8. Feature: target_user_group_counts
target_user_group_counts = (
    hist_enriched
    .group_by(["customer_id", "target_user_group_final"])
    .agg(pl.len().alias("target_user_group_counts"))
)

# 9. Feature 4 (mới): segment_counts theo (customer, category_l1, segment_name)
segment_counts = (
    hist_enriched
    .group_by(["customer_id", "category_l1", "segment_name"])
    .agg(pl.len().alias("segment_counts"))
)

# 10. Xây tập candidate (customer_id, item_id) từ HIST ∪ RECENT
hist_pairs = hist_lf.select(["customer_id", "item_id"]).unique()

```

```

recent_pairs = recent_lf.select(["customer_id", "item_id"]).unique()

candidate_pairs = pl.concat([hist_pairs, recent_pairs]).unique()

# 11. Enrich candidate với thuộc tính item & segment_name đại diện
candidate_enriched = (
    candidate_pairs
    .join(item_attrs, on="item_id", how="left")
    .join(item_segment, on="item_id", how="left")
)

# 12. Join tất cả các bảng count để tạo feature set
features = (
    candidate_enriched
    # brand_counts
    .join(
        brand_counts,
        on=["customer_id", "brand_final"],
        how="left",
    )
    # age_counts
    .join(
        age_counts,
        on=["customer_id", "age_bucket_final"],
        how="left",
    )
    # category_counts
    .join(
        category_counts,
        on=["customer_id", "category"],
        how="left",
    )
    # segment_counts – join theo (customer_id, category_l1, segment_name)
    .join(
        segment_counts,
        on=["customer_id", "category_l1", "segment_name"],
        how="left",
    )
    # target_user_group_counts
    .join(
        target_user_group_counts,
        on=["customer_id", "target_user_group_final"],
        how="left",
    )
    # fill null = 0 cho tất cả count
    .with_columns([
        pl.col("brand_counts").fill_null(0),
        pl.col("age_counts").fill_null(0),
        pl.col("category_counts").fill_null(0),
        pl.col("segment_counts").fill_null(0),
        pl.col("target_user_group_counts").fill_null(0),
    ])
)

# 13. Tạo Label Y từ RECENT: (customer_id, item_id) có giao dịch trong RECENT ->
Y=1
labels = recent_pairs.with_columns(
    pl.lit(1).alias("Y")
)

feature_label_lf = (

```

```

features
.join(labels, on=["customer_id", "item_id"], how="left")
.with_columns(
    pl.col("Y").fill_null(0).cast(pl.Int8)
)
.select([
    "customer_id",
    "item_id",
    "brand_counts",
    "age_counts",
    "category_counts",
    "segment_counts",
    "target_user_group_counts",
    "Y",
])
)

return feature_label_lf

```

Kết quả thu được:

shape: (5, 6)

customer_id	item_id	brand_counts	age_counts	category_counts	Y
i32	str	u32	u32	u32	i8
3818887	"71670000000002"	2	14	2	0
3498899	"01800000000011"	16	0	1	0
6062797	"46970000000002"	6	2	1	0
1936503	"24830000000004"	4	2	1	0
5939703	"54200000000003"	5	24	5	0

- Dựa trên kết quả thu được sau khi thực hiện bài tập, nhóm đã quán và nhận thấy một số điểm như sau:
 - o Dòng dữ liệu có giá trị brand_counts = 2, age_counts = 14, category_counts = 2 nhưng Y = 0
 - o Hoặc trường hợp brand_counts = 5, age_counts = 24, category_counts = 5, tuy nhiên Y vẫn bằng 0

=> 3 đặc trưng trên chưa đủ để phản ánh được toàn bộ khả năng mà người dùng có mua sản phẩm đó hay không.

Thực tế thì hành vi mua hàng của người dùng còn phụ thuộc vào nhiều yếu tố khác như:

- Phân khúc giá của sản phẩm
- Nhóm đối tượng sử dụng
- Mức độ phổ biến
-

Do đó, nhóm quyết định sẽ bổ sung thêm một số đặc trưng để cải thiện khả năng phân biệt và giúp mô hình học tốt hơn

2. Bổ sung thêm một số đặc trưng:

Sau giai đoạn đầu xây dựng ba đặc trưng cơ bản (brand_counts, age_counts, category_counts), nhóm tiếp tục bổ sung thêm ba đặc trưng mở rộng gồm:

- **segment_counts**: số lần khách hàng mua sản phẩm thuộc cùng phân khúc giá
- **target_user_group_counts**: số lần mua sản phẩm dành cho cùng nhóm đối tượng sử dụng (bé trai, bé gái, sơ sinh...)
- **origin_counts**: số lần mua sản phẩm cùng nguồn gốc (nội địa, nhập khẩu)
- Ngoài ra, nhóm còn bổ sung thêm một đặc trưng quan trọng sau này là `time_since_last_purchase_in_B_category`. Feature này đo thời gian (tính bằng ngày) kể từ lần cuối khách hàng mua một sản phẩm thuộc cùng category với sản phẩm mà mô hình cần dự đoán. Đây là một tín hiệu quan trọng vì hành vi mua hàng thường có chu kỳ sử dụng. Ví dụ: nếu một khách hàng vừa mua tã cách đây vài ngày, họ ít có khả năng mua tiếp trong tháng kế tiếp, vì sản phẩm chưa dùng hết. Ngược lại, nếu khách hàng đã 30–40 ngày không mua lại, thì khả năng cao họ đang đến hạn mua mới. Do đó, `time_since_last_purchase_in_B_category` giúp mô hình hiểu được nhu cầu thực tế theo thời điểm, từ đó dự đoán chính xác hơn việc khách hàng có khả năng mua lại sản phẩm thuộc nhóm đó hay không.

1. Lấy thuộc tính item (đặc biệt là category)

```
item_cat = ( item_df[["item_id", "category"]] )
```

2. Lấy giao dịch HIST (dùng lại begin_hist / end_hist từ cell trên)

```
hist_df = ( purchase_df .filter( pl.col("created_date").is_between(begin_hist, end_hist,  
closed="both") ) .join(item_cat, on="item_id", how="left") )
```

3. Tính ngày mua gần nhất theo (customer_id, category)

```
last_cat_purchase = ( hist_df .group_by(["customer_id", "category"]) .agg(  
pl.col("created_date").max().alias("last_purchase_date") ) )
```

4. Join vào feature_label_df để align theo (customer, item)

```
feature_label_df = ( feature_label_df .join(item_cat, on="item_id", how="left")  
  
    .join(last_cat_purchase, on=["customer_id", "category"], how="left") .with_columns(  
    ( pl.lit(end_hist) - pl.col("last_purchase_date")) .dt.total_days()  
    ).alias("time_since_last_purchase_in_B_category") ) .with_columns(  
    pl.col("time_since_last_purchase_in_B_category").fill_null(9999) )  
  
    .drop(["last_purchase_date", "category", "category_right",  
    "last_purchase_date_right"] )  
  
    )
```

```
feature_label_df = feature_label_df.select( [col for col in feature_label_df.columns if  
col != "Y"] + ["Y"] )
```

```
feature_label_df
```

shape: (22, 646, 776, 9)

customer_id	item_id	brand_counts	age_counts	category_counts	segment_counts	target_user_group_counts	origin_counts	Y
i32	str	u32	u32	u32	u32	u32	u32	i8
3513571	"1439000000004"	8	35	10	0	41	7	0
5477989	"3797000000002"	0	0	0	0	0	0	1
6600697	"2117000000010"	1	2	1	21	10	7	0
7413600	"0176000000001"	1	0	1	32	7	0	0
7497617	"0007090000157"	1	6	1	0	3	1	0
...	...	...	...	...	...	...	...	...
5884872	"1387000000025"	2	12	2	100	4	15	0
4767483	"2344000000070"	1	3	1	8	2	1	0
4090194	"6472000000003"	1	2	1	13	2	3	0
7182180	"4512000000002"	3	21	3	0	22	1	0
5025809	"5490000000007"	1	18	1	32	9	4	0

shape: (22, 646, 776, 9)

customer_id	item_id	brand_counts	age_counts	category_counts	segment_counts	target_user_group_counts	time_since_last_purchase_in_B_category	Y
i32	str	u32	u32	u32	u32	u32	i64	i8
6744393	"0037000000032"	1	23	1	3	20	149	0
5915271	"6548000000001"	1	6	1	0	18	116	0
4349523	"3901000000005"	1	12	1	16	7	235	0
7144747	"4280000000030"	20	6	3	0	50	4	0
6751985	"1370000000014"	2	33	2	0	27	69	0
...	...	...	...	...	...	...	...	...
6285344	"1370000000013"	12	39	12	63	46	24	0
7360963	"2700000000002"	1	11	1	2	3	41	0
7244038	"3500000000008"	6	21	2	2	15	200	0
8105309	"3787000000005"	0	0	0	0	0	9999	1
3307365	"6501000000006"	1	6	1	11	4	59	0

Tuy nhiên, khi quan sát bảng kết quả, nhóm nhận thấy việc bổ sung ba đặc trưng mới vẫn chưa đủ để mô hình phân biệt tốt giữa các trường hợp $Y = 1$ và $Y = 0$, chẳng hạn như:

- Có dòng mà tất cả các đặc trưng từ X_1 đến X_6 đều bằng 0, nhưng nhãn Y vẫn bằng 1.
- Hoặc X_1 đến X_6 rất cao (2-12-2-100-4-15) nhưng nhãn Y vẫn ra là 0.

=> Các đặc trưng hiện tại chưa đủ sâu để mô tả, phản ánh được hành vi mua hàng của khách hàng.

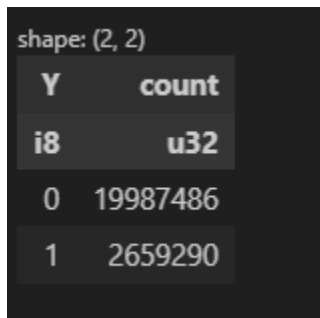
Với sự tò mò dữ liệu, nhóm đang định hướng sẽ xem xét thêm một số dữ kiện khác như:

- Vòng đời sử dụng sản phẩm (có những sản phẩm 1 hoặc 2 ngày mua 1 lần, hoặc có những sản phẩm thì 3-4 tháng mới mua 1 lần)
- Độ phổ biến của sản phẩm trên thị trường
- Khả năng co-purchase (mua cùng) với các sản phẩm khác
- Và nhiều dữ kiện khác nữa.

Nhóm sẽ tiếp tục tìm hiểu và bổ sung thêm đặc trưng trong tương lai để hoàn thành phần này.

Định hướng xử lý negative sample

Sau khi gán nhãn Y cho toàn bộ các cặp (customer_id, item_id), nhóm thực hiện thống kê phân bố nhãn và thu được kết quả như hình bên dưới.



```
shape: (2, 2)
```

Y	count
0	19987486
1	2659290

Qua quan sát, số lượng nhãn 0 lớn hơn nhãn 1 là rất nhiều (gấp khoảng 8 lần). Điều này cho thấy dữ liệu huấn luyện đang bị mất cân bằng, ảnh hưởng đến khả năng phân loại của mô hình (như Mô hình học lệch về phía dự đoán $Y = 0$, Khả năng phân biệt positive kém, Chi phí tính toán tăng cao,...) .

Để khắc phục, nhóm dự định áp dụng negative sampling có chọn lọc nhằm chỉ giữ lại các negative sample hữu ích và loại bỏ những negative sample không cần thiết.

Định hướng xử lý cụ thể như sau:

- Sau khi bổ sung và hoàn thiện đầy đủ các đặc trưng cho tập train, nhóm sẽ xác định ngưỡng lọc negative dựa trên một hoặc nhiều tiêu chí đặc trưng.
- Những mẫu âm có giá trị đặc trưng quá thấp, gần như không mang thông tin (ví dụ: tất cả $X_1 \dots X_6$ đều bằng 0), sẽ được loại bỏ vì khả năng mô hình học được điều gì từ các mẫu này là rất thấp.
- Những negative sample có ý nghĩa nhưng được gán nhãn 0 sẽ được giữ lại (ví dụ như đã 2 tháng mới mua 1 lần thì có thể tháng 10 khách hàng này sẽ không mua nhưng tháng 11 vẫn có khả năng sẽ mua lại) để giúp mô hình tiếp tục học các negative sample có ý nghĩa này.